

Mikrocomputertechnik 1

Einheit 9: Wrap-Up & C-to-Assembly

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 38

Wrap-Up & C-to-Assembly

Folie

2 / 38

Der Kreis schließt sich

- Herzlich willkommen zur letzten regulären Vorlesung in Mikrocomputertechnik 1
- Intensives Semester: Computer von Grund auf verstanden
- Heute: Hochsprachen (C/C++) final mit RISC-V-Assembler verbinden
- Theorie kurz (ca. 45 Min.), danach ca. 3 Stunden Labor
- Abschluss: Mini-Projekt und gezielte Klausurvorbereitung

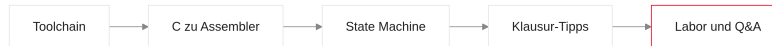


Notizen

Finale: Transistoren, Pipelining, OS. Heute der Bogen von C zum Assembler. Danach Labor und Klausurfragen.

Agenda für den heutigen Tag

1. Der Übersetzungsprozess — klassische C/C++-Toolchain
2. C-to-Assembly — `if`, `while`, `for`
3. Mini-Projekt (State Machine) — Briefing in Ripes
4. Klausurvorbereitung — Organisatorisches und Aufgabentypen
5. Q&A & Labor — freies Coden und Fragen



Notizen

Kurz: Wer erzeugt Maschinencode? Compiler-Muster für Kontrollfluss; dann FSM-Labor und Klausurformate.

Rückblick auf das Semester

- Einheit 1: Logikgatter und Transistoren
- Einheiten 2–4: RISC-V-Assembler und ABI
- Einheiten 5–6: Datenpfad und Control Unit
- Einheit 7: Pipelining — Durchsatz steigern
- Einheit 8: I/O und Syscalls — Außenwelt und OS

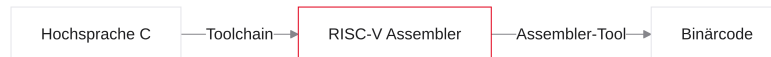
Die Blackbox Computer ist entschlüsselt.
Auf Hardware-Ebene bleiben keine Geheimnisse.

Notizen

Treppe der Abstraktion: Transistoren, ISA/ABI, Datenpfad, Pipeline, I/O.

Die letzte Lücke

- Wochenlang Assembler in Venus und Ripes
- Industrie: große Programme in C, C++, Rust, ...
- Frage: Wie wird `printf("Hallo");` ausführbarer Maschinencode (viele Wörter / Syscalls)?
- Antwort: die Toolchain (Werkzeugkette)



Notizen

Brücke zwischen C und dem 5-Stufen-Prozessor: die Toolchain.

Die Toolchain

Der Übersetzungsprozess

- `riscv64-unknown-elf-gcc -march=rv32i -mabi=ilp32` erzeugt RISC-V-Code
- Ein nacktes `gcc main.c` auf dem Host tut das nicht
- Ausgabearten: Assemblertext, Objektdatei mit Relocation, gelinktes ELF
- `printf` braucht eine passende Laufzeit; ein ELF ist kein Venus-Assemblertext
- Output des einen = Input des nächsten



Notizen

Vier Stationen: Preprocessor, Compiler, Assembler, Linker.

1. Preprocessor und 2. Compiler

- Preprocessor: Text-Ersetzer für # . . . (#include, #define)
- Header-Text wird eingefügt; Makros expandiert
- Compiler: Syntax/Semantik, Optimierung; Ausgabe: RISC-V-Assembler (.s)

```
int a = 5;
```

```
li s0, 5
```

Notizen

Preprocessor = Textersetzung. Compiler = Kern: C wird zu Assembler.

3. Assembler

- Eingabe: Text mit add, lw, beq
- Ausgabe: 32-Bit-Maschinencode (Object-File .o)
- Nutzt Instruktionsformate R/I/S/B/J (Einheit 2)
- Noch nicht ausführbar: Adressen oft noch offen

```
Text:    add t0, t1, t2  
Hex:    0x007302B3
```

Notizen

Assembler packt Opcode/rs1/rs2/rd laut Handbuch. Object-Dateien fehlen oft noch finale Adressen.

4. Linker

- Compiler kennt die finale Adresse von printf oft nicht
- jal/lw-Ziele können Lücken haben
- Linker: alle .o + Libraries (z. B. libc) zusammenkleben
- Finale Adressen eintragen
- Output: Executable (.elf / .exe)

Notizen

Erst der Linker macht das Programm ladbar.

Folie

11 / 38

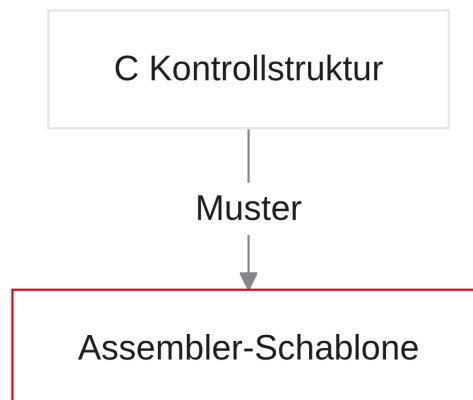
C-to-Assembly: Kontrollstrukturen

Folie

12 / 38

Muster des Compilers

- Wie wird übersetzt?
- Rolle: Compiler-Hut aufsetzen
- `if`, `while`, `for` - Schablonen
- Relevant für Klausur, Reverse Engineering, Debugging



Notizen

Kopf-Übersetzung der gängigen C-Muster.

Variablen und Register

- Lokale Variablen: Compiler bevorzugt schnelle Register
- ABI: oft `s0-s11` für langlebige Lokale
- Register Pressure: Spilling auf den Stack (`sw/lw`)

```
int a = 5;  
int b = 10;  
int c = a + b;
```

```
li s0, 5  
li s1, 10  
add s2, s0, s1
```

Notizen

Solange Platz: Variablen in Registern. Spilling kostet Performance.

Das if-Statement (C-Code)

- Klassisches If-Else
- Vergleich a und b
- Gleich: Block 1; sonst Block 2

```
int a = 5;  
int b = 10;  
int c;  
  
if (a == b) {  
    c = 1;  
} else {  
    c = 2;  
}
```

Notizen

Bedingungs-Umkehrung aus Einheit 3: Assembler fragt oft das Gegenteil.

Das if-Statement (Assembler)

- C: `a == b` → Assembler oft `bne` (Gegenteil)
- Ungleich: zum Else springen
- Am Ende des If-Blocks: `j End` - sonst Rutschen in Else

```
li s0, 5
li s1, 10
bne s0, s1, Else_Block

If_Block:
li s2, 1
j End

Else_Block:
li s2, 2

End:
```

Notizen

Klassischer Klausurfehler: fehlender Jump nach dem If-Block.

Die while-Schleife (C-Code)

- Kopfgesteuert: Bedingung vor dem Körper
- `i` hochzählen, bis 10

```
int i = 0;

while (i < 10) {
    i++;
}
```

Notizen

Ist die Bedingung anfangs falsch, läuft der Körper kein Mal.

Die while-Schleife (Assembler)

- Label While_Start am Kopf
- C: $i < 10 \rightarrow$ Assembler oft bge (Abbruch)
- Am Ende: j While_Start

```
li s0, 0
li t0, 10

While_Start:
    bge s0, t0, While_End
    addi s0, s0, 1
    j While_Start

While_End:
```

Notizen

Kreis: Prüfung, Body, Rücksprung.

Die for-Schleife (C-Code)

- Syntaktischer Zucker für while
- Init, Bedingung, Inkrement in einer Zeile
- Beispiel: Array-Summe

```
int sum = 0;
int arr[10] = { /* ... */ };

for (int i = 0; i < 10; i++) {
    sum = sum + arr[i];
}
```

Notizen

Standard zum Iterieren über Arrays.

Die for-Schleife (Assembler)

1. Initialisierung vor dem Label
2. Bedingung direkt danach
3. Body
4. Inkrement
5. Rücksprung

```
li s0, 0          # sum = 0
li s1, 0          # i = 0
li t0, 10
la s2, arr

For_Loop:
    bge s1, t0, For_End
    # Body: siehe nächste Folie
    addi s1, s1, 1
    j For_Loop

For_End:
```

Notizen

Blaupause jeder For-Schleife in Assembler.

Arrays: `arr[i]` in Hardware

- C: `arr[i]` = i-tes Element
- CPU: Bytes - int = 4 Byte
- Adresse = Base + ($i \times 4$) oft als `slli` um 2

```
slli t1, s1, 2      # i * 4
add  t2, s2, t1      # Base + Offset
lw   t3, 0(t2)
add  s0, s0, t3      # sum += arr[i]
```

Notizen

Klassischer Klausurfehler: Offset ohne Multiplikation mit 4.

Funktionen: ABI, a0 und ret

- Parameter in a0, a1, ... (ABI, Einheit 4)
- C-return = Wert nach a0, dann ret (Sprung über ra)
- Leaf-Beispiel unten; nicht-leaf braucht Stack-Prolog/Epilog (Klausur-Typ 2)

```
int calc(int a) {  
    return a * 2;  
}
```

Fragment, kein vollständiges Programm:

```
calc:  
    slli a0, a0, 1  
    ret
```

Aufrufer separat, mit Exit, damit calc nicht per Fall-through erneut läuft

Notizen

Leaf-Funktionen können ohne Frame auskommen. Ruft die Funktion weiter auf oder nutzt s-Register, greifen Prolog/Epilog aus Einheit 4.

Compiler-Output in Echt (Godbolt)

- Lehrübersetzung und Compileroutput unterscheiden
- -O0 verspricht keine feste Belegung von s0/s1
- -O3 verspricht keinen bestimmten Pipelineplan
- Konstanten wie $a = 5$; $b = 10$; $c = a + b$ können vollständig entfallen
- Compiler Explorer: Version, -march=rv32i, -mabi=ilp32 dokumentieren

Notizen

Release-Builds optimieren aggressiv für Pipeline und Latenz.

Warum Assembler lernen?

1. Maschinenverständnis: Caches, Stalls, Spilling
2. Embedded: Bootloader, ISR, Treiber - C und Assembler gemischt
3. Security / Reverse Engineering: oft nur Disassembly

Assembler ist selten für große Apps.
Assembler ist das Röntgengerät des Informatikers.

Notizen

Wer RISC-V versteht, schreibt besseres C und liest Maschinencode.

Labor & Klausur

Übergang in die Praxisphase

- Formaler Vorlesungsteil beendet
- Ca. 3 Stunden Praxis
- Abschlussprojekt: Finite State Machine (FSM)
- Danach: Klausur-Formalia und Aufgabentypen



Notizen

FSM bündelt Kontrollfluss, I/O und Logik des Semesters.

Projekt: Die State Machine

- Finite State Machine (FSM): zu jedem Zeitpunkt genau ein Zustand
- Eingaben (Taster) → Übergänge (Transitions)
- Fundament: Ampeln, Spiele, Protokolle, Steuerungstechnik

Notizen

FSM merkt sich den Status; Sensorereignisse triggern Zustandswechsel.

Code-Schloss

- S0 gesperrt, LED rot; S1 nach UP, LED gelb; S2 nach DOWN, LED blau; S3 offen, LED grün
- Sequenz der Ereignisse: UP, DOWN, RIGHT
- Halten erzeugt höchstens ein Ereignis; Release trennt die Drücke
- Falsche Taste und Reset nach S0; ohne Taste und bei mehreren Tasten bleiben
- S3 bleibt offen bis Reset

Code-Schloss · Ereignisse, nicht Pegel



Trace: UP, Release, DOWN, Release, RIGHT, Release * S0, S1, S2, S3.

Halten löst kein zweites Ereignis aus. Falsche Taste und Reset führen nach S0.

Keine Taste: Zustand bleibt. Mehrere Tasten gleichzeitig: kein gültiges Ereignis, Zustand bleibt.

Offener Zustand S3 bleibt, bis Reset. Entprellung erzeugt das Ereignis, die FSM sieht es nur einmal.

Selbst gezeichnetes Lehrdiagramm. LED-Farbe gehört zum Zustand, das Ereignis zur Kante.

Abbildung 1: Zustände des Code-Schlusses

Trace: UP, Release, DOWN, Release, RIGHT, Release führt von S0 über S1 und S2 nach S3.

Notizen

Entprellung erzeugt das Ereignis. Die FSM sieht einen Druck nur einmal, auch wenn der Pegel gehalten wird.

Umsetzung in Assembler

- Zustandsvariable in sicherem Register (z. B. `s0`)
- Endlosschleife: Turm aus `beq` zu State-Labels
- Im State: Arbeit (LED, Taster), dann `s0` ändern, `j Loop`

```
li s0, 0
Loop:
    beq s0, zero, State_0
    li t0, 1
    beq s0, t0, State_1
    j State_0          # Default / unbekannt

State_0:
    # LED, Taster, ggf. s0 = 1
    j Loop

State_1:
    # LED, Taster, ggf. s0 = 2
    j Loop
```

Notizen

Skizze: Verteilerkopf oben, je State ein Label. Unbekannte Zustände sicher auf State 0.

Laboraufgabe: Das Code-Schloss

- Ripes: D-Pad + LED-Matrix
- Geheimcode: OBEN, UNTEN, RECHTS
- Start: LED rot (State 0)
- Richtig: State 1 gelb → State 2 blau → grün (offen)
- Falsche Taste: sofort zurück auf State 0 (rot)

Tipp: Entprellung (Debouncing) aus Einheit 8!
Sonst rauscht ein Klick durch alle States.

Notizen

Perfektes FSM-Labor: I/O + Kontrollfluss + Entprellung.

Die Klausur (Organisatorisches)

- Schriftliche Klausur (Orientierung: ca. 90 Minuten)
- Hilfsmittel: RISC-V Green Card / Reference Sheet liegt bei
- Kein Auswendiglernen von Opcodes oder Register-Nummern
- Fokus: Verständnis, Code-Analyse, Problemlösung

Notizen

Tabellen lesen können ja; stumpfe Opcodes nein.

Klausur-Typ 1: Code-Analyse (Tracing)

- Assembler-Snippet + Startwerte
- Rolle: menschlicher Simulator
- Frage z. B.: Wert in `t2` nach Ablauf?

```
li t0, 5
li t1, 3
sub t2, t0, t1    # t2 = 2
slli t2, t2, 1    # *2
# Ergebnis: t2 = 4
```

Notizen

Wie Venus Step-by-Step - Registerzeilen auf dem Schmierblatt.

Klausur-Typ 2: Code schreiben

- C → RISC-V übersetzen (Labor-Niveau)
- Beispiele: Summe 1..10; Funktion mit ABI (Prolog/Epilog)
- Register-Konventionen penibel einhalten

```
int summe = 0;
for (int i = 0; i < 5; i++) {
    summe += i;
}
```

Notizen

For-Schablone + bei Funktionen Stack-Prolog/Epilog.

Folie

33/38

Klausur-Typ 3: Hardware / Datenpfad

- Einheiten 5–6: Single-Cycle-Blockdiagramm
- Datenweg eines `lw` markieren
- Control-Zeile für z. B. `or` ausfüllen

Befehl	ALUSrc	Mem- toReg	RegWrite	Mem- Read	MemWrite	Branch	ALUOp
<code>or</code>	0	0	1	0	0	0	10

- Main Control ist für alle R-Types gleich; `or` vs. `add` entscheidet die ALU Control (`funct3`)

Notizen

Deduktives Tracing aus Einheit 6: Signale ableiten, nicht auswendig lernen.

Folie

34/38

Klausur-Typ 4: Pipelining & Hazards

- Einheit 7: Timing-Raster IF/ID/EX/MEM/WB
- RAW-Hazards erkennen; Forwarding-Pfeile
- Unvermeidbarer Stall (Load-Use)

Befehl	1	2	3	4	5	6	7
<code>lw t0, 0(a0)</code>	IF	ID	EX	MEM	WB		
<code>add t1, t0, t0</code>		IF	ID	stall	EX	MEM	WB

Notizen

Load-Use: eine Bubble (stall). Danach Forwarding MEM→EX, wenn der Wert aus dem RAM kommt (Einheit 7).

Tipps zur Vorbereitung

1. Alte Laborblätter ohne Vorlage neu lösen; Venus Step nutzen
2. Godbolt: kleine C-Snippets, `-O0`, Sprünge analysieren
3. Lerngruppen: erklären, warum `MemtoReg` bei `sw Don't Care (X)` ist

Wie Mathematik: Durchlesen reicht nicht.
Sie müssen Code tippen.

Notizen

Simulator-Fehlermeldungen sind der beste Nachhilfelehrer.

Feedback & Evaluation

- Bitte an der anonymen Evaluation teilnehmen
- Tempo, Erklärungen, Nutzen der Simulatoren?
- QR-Code / Link der Hochschule live in der Vorlesung

Notizen

Feedback verbessert die kommenden Semester.

Das war Mikrocomputertechnik 1

- Fundamentaler Meilenstein erreicht
- Computer als klare, logische Maschine
- Später in C/Java/Python: physikalische Konsequenzen im Blick
- Danke für Mitarbeit - viel Erfolg in der Klausur

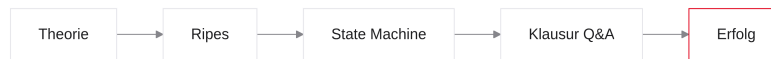
```
# Offizieller Ausstieg (Venus-ABI, nicht Ripes-MMIO):  
li a0, 10  
ecall
```

Notizen

Venus: Service-ID in a0 (Einheit 8). Ripes-Labor nutzt MMIO, keinen Exit-Syscall.

Q&A und Hands-On

- Restliche Zeit: Ihnen
- Ripes: Code-Schloss (FSM)
- Dozent zum Debuggen und für Klausurfragen da
- Labor ist eröffnet



Notizen

Safe bauen, entprellen, offene Fragen klären - viel Erfolg!